Python Interview Questions

For B.Tech Final Year Placement Drives

50 Questions | Concepts + Coding | With Brief Answers

A curated set of frequently asked Python questions covering core language concepts, object-oriented programming, data structures, error handling, and common coding problems asked in campus placement interviews and online assessments.

Section A: Core Python Concepts (Q1 - Q20)

Q1. What is the difference between a list and a tuple in Python?

A: Lists are mutable, tuples are immutable. Lists use more memory and are slightly slower to iterate. Tuples can be used as dictionary keys (if elements are hashable); lists cannot.

Q2. What are Python's mutable and immutable data types?

A: Immutable: int, float, str, tuple, bool, frozenset. Mutable: list, dict, set, bytearray. Immutable objects cannot be changed after creation; any "change" creates a new object.

Q3. Explain the difference between `is` and `==`.

A: `==` checks value equality; `is` checks whether two references point to the same object in memory (identity).

Q4. What is the Global Interpreter Lock (GIL)?

A: The GIL is a mutex in CPython that allows only one thread to execute Python bytecode at a time, even on multi-core systems. It simplifies memory management but limits true parallelism in CPU-bound multithreaded programs (multiprocessing is used to bypass it).

Q5. What are *args and **kwargs?

A: `*args` collects extra positional arguments into a tuple; `**kwargs` collects extra keyword arguments into a dictionary. They allow functions to accept a variable number of arguments.

Q6. Explain list comprehension with an example.

A: A concise way to create lists using a single line of code.

```
squares = [x**2 for x in range(10) if x % 2 == 0]
# [0, 4, 16, 36, 64]
```

Q7. What is the difference between deep copy and shallow copy?

A: A shallow copy (`copy.copy()`) creates a new object but inserts references to the same nested objects. A deep copy (`copy.deepcopy()`) recursively copies all nested objects, so changes to nested data don't affect the original.

Q8. What are Python decorators?

A: A decorator is a function that wraps another function to extend its behavior without modifying its source code, using the `@decorator_name` syntax. Commonly used for logging, timing, and access control.

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print("Before call")
        result = func(*args, **kwargs)
        print("After call")
        return result
    return wrapper

@my_decorator
def greet():
    print("Hello")
```

Q9. What is a generator? How is it different from a normal function?

A: A generator uses `yield` instead of `return` to produce a sequence of values lazily, one at a time, preserving state between calls. It is memory-efficient compared to returning a full list, since values are generated on demand.

Q10. What is the difference between `__init__` and `__new__`?

A: `__new__` is a static method that creates and returns a new instance (allocates memory); `__init__` initializes the already-created instance's attributes. `__new__` runs first.

Q11. Explain Python's memory management model.

A: Python uses automatic memory management with a private heap. Objects are managed via reference counting, and a cyclic garbage collector handles reference cycles that reference counting alone can't clean up.

Q12. What is the difference between `append()` and `extend()` in lists?

A: `append()` adds a single element (even a list) as one item at the end. `extend()` iterates over its argument and adds each element individually to the list.

Q13. What are Python's exception handling keywords?

A: `try` (code that may raise an error), `except` (handles the error), `else` (runs if no exception occurs), `finally` (always runs, used for cleanup), and `raise` (manually raises an exception).

Q14. What is the difference between multithreading and multiprocessing?

A: Multithreading runs multiple threads within the same process sharing memory but is limited by the GIL for CPU-bound tasks. Multiprocessing spawns separate processes each with its own interpreter and memory, allowing true parallelism, better for CPU-bound work.

Q15. What are Python's lambda functions?

A: Anonymous, single-expression functions defined using the `lambda` keyword, commonly used with `map()`, `filter()`, and `sorted()`. Example: `square = lambda x: x*x`

Q16. What is the difference between `@staticmethod` and `@classmethod`?

A: A `@classmethod` receives the class (`cls`) as its first argument and can access/modify class state. A `@staticmethod` receives no implicit first argument and behaves like a plain function placed inside a class for organizational purposes.

Q17. What is monkey patching in Python?

A: Dynamically modifying or extending a class or module at runtime, e.g., adding/replacing a method on an existing class without changing its source code.

Q18. What is the purpose of the `with` statement in Python?

A: It is used for context management (e.g., file handling), ensuring resources are properly acquired and released (like automatically closing a file) even if an exception occurs, via `__enter__` and `__exit__` methods.

Q19. What is the difference between a shallow and deep understanding of Python's pass-by-object-reference?

A: Python passes references to objects by value. For mutable objects, changes made inside a function affect the original object; for immutable objects, rebinding inside a function does not affect the caller's variable.

Q20. What are Python's magic/dunder methods? Give examples.

A: Special methods with double underscores that let objects support built-in operations. Examples: `__init__` (constructor), `__str__` (string representation), `__len__` (length), `__add__` (overload +), `__eq__` (equality).

Section B: Object-Oriented Programming (Q21 - Q30)

Q21. What are the four pillars of OOP and how does Python support them?

A: Encapsulation (bundling data with methods, using naming conventions like `_var` / `__var`), Abstraction (hiding implementation via abstract base classes), Inheritance (deriving classes from a base class), Polymorphism (same interface, different implementations, e.g., method overriding).

Q22. What is method resolution order (MRO) in Python?

A: MRO determines the order in which base classes are searched when executing a method in multiple inheritance. Python uses the C3 linearization algorithm; it can be viewed using `ClassName.__mro__`.

Q23. What is method overloading and does Python support it?

A: Method overloading means defining multiple methods with the same name but different parameters. Python does not support it natively (later definitions override earlier ones); it is emulated using default arguments or `*args`.

Q24. What is method overriding? Give an example.

A: A subclass provides a specific implementation of a method already defined in its parent class.

```
class Animal:
    def sound(self):
        print("Some sound")

class Dog(Animal):
    def sound(self):
        print("Bark")
```

Q25. What is the difference between a class variable and an instance variable?

A: A class variable is shared across all instances of a class. An instance variable is unique to each object and is typically defined inside `__init__` using `self`.

Q26. What is an abstract class in Python?

A: A class that cannot be instantiated directly and typically contains one or more abstract methods that subclasses must implement. Created using the `abc` module and `@abstractmethod` decorator.

Q27. What is encapsulation, and how do you achieve private variables in Python?

A: Encapsulation restricts direct access to an object's data. Python uses naming conventions: a single underscore (`_var`) suggests "protected", a double underscore (`__var`) triggers name mangling to `_ClassName__var`, making it harder to access from outside.

Q28. What is duck typing in Python?

A: A concept where an object's suitability is determined by the presence of certain methods/attributes rather than its actual type — "if it walks like a duck and quacks like a duck, it's a duck."

Q29. What is the difference between composition and inheritance?

A: Inheritance models an "is-a" relationship (a subclass extends a parent class). Composition models a "has-a" relationship (a class contains instances of other classes as attributes), often preferred for flexibility and lower coupling.

Q30. What is operator overloading? Give an example.

A: Redefining the behavior of operators for custom objects using dunder methods.

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)
```

Section C: Data Structures & Built-ins (Q31 - Q40)

Q31. What is the difference between a set and a dictionary?

A: A set is an unordered collection of unique, hashable elements. A dictionary stores unique key-value pairs. Both use hashing internally for O(1) average lookup.

Q32. How does a Python dictionary achieve O(1) average lookup time?

A: It uses a hash table internally. Each key is hashed to compute an index into an underlying array, allowing near-constant time insertion, deletion, and lookup on average (worst case O(n) with many collisions).

Q33. What is the difference between `remove()`, `pop()`, and `del` for lists?

A: `remove(value)` removes the first matching value. `pop(index)` removes and returns the element at a given index (last by default). `del list[index]` deletes an element/slice by position without returning it.

Q34. What are Python's built-in data structures for stacks and queues?

A: A list can act as a stack (`append`/`pop`). `collections.deque` is preferred for queues/stacks because it offers O(1) appends and pops from both ends, unlike a list's O(n) pop(0).

Q35. What is a namedtuple and why use it?

A: A `collections.namedtuple` creates tuple subclasses with named fields, improving code readability by allowing attribute-style access instead of index-based access.

Q36. Explain slicing in Python with examples.

A: Slicing extracts a portion of a sequence using `seq[start:stop:step]`.

```
a = [0,1,2,3,4,5]
a[1:4]    # [1, 2, 3]
a[::-1]   # [5, 4, 3, 2, 1, 0]  -> reversed
a[::2]    # [0, 2, 4]
```

Q37. What is the difference between `map()`, `filter()`, and `reduce()`?

A: `map(func, iterable)` applies a function to every element. `filter(func, iterable)` keeps elements where func returns True. `reduce(func, iterable)` (from functools) cumulatively combines elements into a single value.

Q38. What is a Python dictionary comprehension?

A: A concise way to build dictionaries in one line: `{k: v for k, v in pairs}`. Example: `{x: x**2 for x in range(5)}`.

Q39. What is the difference between `__str__` and `__repr__`?

A: `__str__` returns a readable, user-friendly string representation (used by `print()`). `__repr__` returns an unambiguous representation intended for developers/debugging, ideally one that could recreate the object.

Q40. How do you handle missing keys in a dictionary safely?

A: Use `dict.get(key, default)` to avoid a `KeyError`, or `collections.defaultdict(factory)` to auto-initialize missing keys with a default value.

Section D: Coding Problems (Q41 - Q50)

Q41. Write a program to check if a string is a palindrome.

```
def is_palindrome(s):
    s = s.lower().replace(" ", "")
    return s == s[::-1]

print(is_palindrome("Madam"))  # True
```

Q42. Write a program to find the factorial of a number using recursion.

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))  # 120
```

Q43. Write a program to check if a number is prime.

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
```

Q44. Write a program to find duplicate elements in a list.

```
def find_duplicates(lst):
    seen, dupes = set(), set()
    for item in lst:
        if item in seen:
            dupes.add(item)
        seen.add(item)
    return list(dupes)
```

Q45. Write a program to reverse a linked list (conceptual/array-based).

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

def reverse_list(head):
    prev = None
    while head:
        nxt = head.next
        head.next = prev
        prev = head
        head = nxt
    return prev
```

Q46. Write a program to count the frequency of words in a sentence.

```
from collections import Counter

def word_freq(sentence):
    return Counter(sentence.lower().split())

print(word_freq("the cat sat on the mat"))
# Counter({'the': 2, 'cat': 1, 'sat': 1, 'on': 1, 'mat': 1})
```

Q47. Write a program to implement binary search.

```
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

Q48. Write a program to find the second largest number in a list without using sort().

```
def second_largest(lst):
    first = second = float('-inf')
    for num in lst:
        if num > first:
            first, second = num, first
        elif first > num > second:
            second = num
    return second
```

Q49. Write a program to check if two strings are anagrams of each other.

```
def is_anagram(s1, s2):
    return sorted(s1.replace(" ", "").lower()) == \
           sorted(s2.replace(" ", "").lower())

print(is_anagram("listen", "silent"))  # True
```

Q50. Write a program to generate the Fibonacci sequence up to n terms using a generator.

```
def fibonacci(n):
    a, b = 0, 1
    for _ in range(n):
        yield a
        a, b = b, a + b

print(list(fibonacci(8)))
# [0, 1, 1, 2, 3, 5, 8, 13]
```

Tip: In interviews, along with the correct code, be ready to explain the time and space complexity of your solution and discuss edge cases (empty input, negative numbers, duplicates, etc.).